

Ejercicios de listas

Ejercicio 1

Escribir un programa que cree un vector de tamaño 100 y que contenga los 100 primeros números naturales y luego los imprima separados por espacios.

1. Primero escribir el bucle de impresión usando `size()` y el operador `[]`.
2. Escribir el bucle usando un iterador.

Ejercicio 2

Escribir un programa que lea una lista de palabras por la entrada estándar separadas por espacios. Eliminar las palabras que contengan la vocal 'a' e imprimir la lista.

Ejercicio 3

Escribir un programa que cree dos listas de 10000 números aleatorios entre 0 y 100. Recorrer las listas comparando los elementos que ocupan la misma posición entre sí y almacenando el mayor de ellos en una tercera lista. Finalmente recorrer esta lista para calcular la media de los números (es resultado debería estar en torno a 66).

Ejercicio 4

Crear una lista de 20 elementos con números aleatorios del 1 al 10. A continuación iterar y en cada posición replicar el número que aparece tantas veces como su valor. Para una array de 4 elementos sería el resultado: 3 6 2 7 = 3 3 3 6 6 6 6 6 6 2 2 7 7 7 7 7 7 7 7. Imprimir la lista resultante de manera inversa.

Ejercicio 5

Escribir una función que ordene los elementos de una lista por el algoritmo de la burbuja. Este algoritmo consiste en recorrer el vector de adelante a atrás comparando un elemento y su siguiente. Cuando el elemento anterior es mayor que el siguiente los debe intercambiar. Tras la primera iteración, la lista contiene el elemento mayor de todos en la última posición. El proceso se repite pero en vez de recorriendo toda la lista, no se considera la última posición pues ya está ordenada. El bucle se repite tantas veces como haga falta, hasta que toda la lista esté ordenada.

Ej:	vector inicial	84 34 12 9 102 45 7 33 17
	tras la primera iteración	34 12 9 84 45 7 33 17 102
	tras la segunda iteración	12 9 34 45 7 33 17 84 102

Ejercicio 6

Escribir una función que tome dos argumentos, un número `n` y una lista. Al volver de la función, la lista deberá contener todos los números primos menores que `n`. La lista pasada como argumento puede contener números, por lo que lo primero a comprobar será si el último número almacenado (si hay alguno) es menor que `n`.

Ejercicio 7

Escribir una función que tome dos listas de listas de Double, compruebe que representan dos matrices de tamaño $(n \times m)$ y $(m \times q)$ respectivamente y devuelva una lista de listas de Double que sea de tamaño $(n \times q)$ con el producto de las matrices.

Ejercicio 8

Escribir un programa que genere una lista con los números impares de 1 a 99 y otra con los números pares de 2 a 100. A continuación mezclar ambas listas con la operación merge sobre la primera lista. Finalmente comprobar con un bucle que la lista resultado contiene todos los números de 1 a 100. No hace falta imprimir ninguna lista, basta con comprobar la corrección del resultado.

Ejercicio 9

Escribir un programa que cree una lista con los números del 1 al 20. A continuación imprimir la lista al revés usando un iterador inverso.

Ejercicio 10

Escribir un programa que genere una lista con 500 números aleatorios del 1 al 50 y que elimine todos aquellos que aparecen repetidos. Se deben eliminar todas las repeticiones de la lista, sin dejar ninguna.

Llevar a cabo el mismo proceso pero ordenando la lista previamente con sort. Eso debería simplificar el número de operaciones necesarias.

Ejercicio 11 (OJO!! Muy complicado)

Escribir una función “partir” que dada una lista de números reales y un número (llamado pivote), devuelve dos listas una con los elementos mayores o iguales a ese número y otra con los menores. La lista origen debe quedar vacía tras el proceso y asegurarse de que no se copian los elementos si no que se mueven de una lista a otra. El prototipo sugerido para esta función es:

```
void partir(List<Double> lista_origen, double pivote, List<Double> izquierda, List<Double> derecha);
```

A continuación usar la función anterior para implementar una función “ordenar” que ordene una lista de números reales. La idea es la siguiente: La función recibiría una lista por referencia. Si la lista está vacía o sólo tiene un elemento, no haría nada. De lo contrario cogería el primer elemento de la lista y lo usaría como pivote para llamar a la función “partir”. Con las listas resultado se llama recursivamente (es decir, la función se llama a sí misma) a la función de “ordenar”. A continuación las listas (ya ordenadas por la llamada recursiva) se mezclan en la lista original.

Generar una lista de 100 números aleatorios y ordenarla con la función “ordenar”. Comprobar, comparando el resultado obtenido con una copia de la lista ordenada por el método sort, que los elementos de ambas listas son los mismos.